

Arquitectura de Datos y Plataforma Analitica RadioShack

ETL, modelo dimensional, API segura, dashboard y despliegue contenerizado

Tipo de documento	Informe de arquitectura para proyecto de titulacion
Alcance	Negocio, datos, aplicaciones, tecnologia, seguridad y costos
Estado evaluado	Prototipo funcional local, previo a produccion
Fecha	28 de junio de 2026

Tesis que sostiene la arquitectura

Separar la fuente transaccional de la carga analitica permite consultar indicadores con seguridad, trazabilidad y menor impacto sobre RadioShack, manteniendo una ruta de crecimiento proporcional al uso real.

Resumen ejecutivo

El proyecto implementa una arquitectura de datos para extraer ventas desde RadioShack/ICG en modo solo lectura, transformar y validar la informacion, cargarla en un almacen PostgreSQL y exponer indicadores mediante una API FastAPI consumida por React, Power BI o Excel. La solucion desacopla el uso analitico del sistema transaccional y evita ejecutar consultas de dashboard directamente sobre la base operativa.

El prototipo tiene una base tecnica defendible: ETL incremental por fecha, reproceso idempotente, staging, dimensiones y tabla de hechos, controles fuente-staging-fact, autenticacion JWT, RBAC, API keys con hash y scopes, migraciones, pruebas unitarias y contenedores. No obstante, aun no debe declararse listo para produccion ni certificado bajo ISO/IEC 27001. Faltan controles operativos como TLS, backups probados, CI/CD, monitoreo, rate limiting, gestion de refresh tokens, cierre del registro publico y pruebas de seguridad.

Recomendacion ejecutiva

Para la sustentacion, presentar el despliegue como arquitectura objetivo y demostrar localmente el flujo completo. Un piloto remoto en un unico servidor Linux de 4 GB es razonable si el volumen diario cabe en memoria; 8 GB ofrece mayor margen para pandas y PostgreSQL. Kubernetes no se justifica en esta etapa.

Contenido

1. Problema, objetivos y valor para el negocio
2. Arquitectura de datos y aplicaciones
3. Decisiones tecnologicas y alternativas
4. Seguridad, normas y brechas
5. Arquitectura de despliegue
6. Capacidad, costos y rentabilidad
7. Escalabilidad y roadmap
8. Plan de sustentacion y preguntas del jurado

1. Problema, objetivos y valor para el negocio

1.1 Situacion problematica

La informacion de ventas reside en una fuente transaccional que solo puede consultarse. Utilizarla directamente para reportes incrementa el acoplamiento, repite calculos, afecta el rendimiento operacional y dificulta conservar reglas de negocio consistentes. Ademas, los campos de fecha-hora no

siempre tienen una hora confiable, por lo que el proceso debe trabajar por fecha de negocio y reprocesar una ventana controlada.

- Consultas analíticas y operacionales compiten por recursos si se ejecutan en la misma fuente.
- Los indicadores pueden diferir si cada herramienta aplica filtros y transformaciones propias.
- No existe una capa única para calidad, trazabilidad, seguridad y consumo externo.
- Una base cloud básica puede volverse costosa si recibe consultas repetidas y no agregadas.

1.2 Objetivo general

Disenar e implementar una arquitectura de datos contenerizada que integre ventas de RadioShack en un almacén analítico, aplique controles de calidad y ofrezca indicadores seguros y reutilizables para una aplicación web, Power BI y Excel.

1.3 Objetivos específicos

1. Extraer datos sin modificar la fuente y con filtros por fecha.
2. Normalizar canales, productos, tiendas y clientes mediante un ETL reproducible.
3. Validar conteos, cantidades y montos entre fuente, staging y hechos.
4. Modelar ventas en un esquema dimensional optimizado para lectura.
5. Exponer KPIs, tendencias, ABC, RFM y rankings con filtros y límites.
6. Proteger usuarios e integraciones con mecanismos de autenticación distintos.
7. Definir una ruta de despliegue, seguridad, costo y escalamiento medible.

1.4 Capacidades de negocio habilitadas

Capacidad	Resultado esperado	Indicadores
Integración de ventas	Datos diarios consistentes sin alterar RadioShack	Filas, monto y cantidad conciliados
Análisis comercial	Vision por fecha, canal, tienda y producto	Ventas, ticket, unidades, documentos
Gestión de portafolio	Priorización de productos por periodo	ABC, top y bajo movimiento
Análisis de clientes	Segmentación de comportamiento	RFM, frecuencia y valor
Consumo multicanal	Una API para web, Power BI y Excel	JWT y API keys con scope

Tabla 1. Capacidades de negocio y resultados medibles.

2. Arquitectura de datos y aplicaciones

Arquitectura logica y flujo de datos

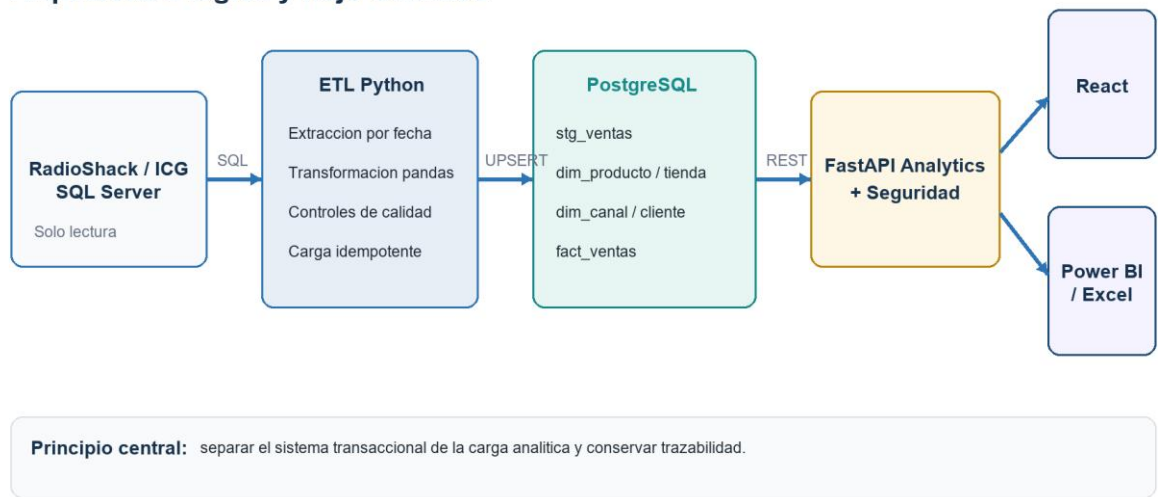


Figura 1. Arquitectura logica implementada y flujo de datos.

2.1 Estilo arquitectonico

La solucion combina un pipeline ETL con un almacen dimensional y un monolito modular orientado a API. El backend separa routers, servicios, repositorios, schemas y modelos. Esta eleccion mantiene una complejidad adecuada para un proyecto de titulacion y permite evolucionar modulos sin introducir microservicios prematuros.

Capa	Responsabilidad	Evidencia actual
Fuente	SQL Server/ICG consultado en modo solo lectura	Repositorio ICG con consultas parametrizadas
ETL	Extraccion, transformacion, calidad y carga	Servicios por ventas, productos y tiendas
Staging	Persistencia temporal y diagnostico	stg_ventas por fecha
Warehouse	Dimensiones y hechos para consulta	fact_ventas y dimensiones conformadas
API	Seguridad y contratos analiticos	FastAPI, Pydantic, JWT, RBAC, API keys
Presentacion	Dashboard y consumo externo	React, Power BI y Excel

Tabla 2. Capas y responsabilidades.

2.2 Modelo dimensional

La tabla fact_ventas representa lineas de venta agregadas por documento, producto, precio y fecha. Se relaciona con dim_producto, dim_tienda, dim_canal y dim_cliente. Este esquema en estrella reduce joins complejos para los consumidores y centraliza codigos de negocio.

- Grano declarado: linea analitica de venta por fecha, documento, producto y precio.

- Dimensiones conformadas: producto, tienda, canal y cliente.
- Medidas: cantidad, precio, descuento, total e IVA.
- Indices compuestos: fecha con producto, cliente, tienda y canal.
- Pendiente recomendado: migrar montos de float a Numeric/Decimal.

2.3 Calidad e idempotencia

Cada fecha se elimina y reconstruye en staging y hechos. Esta estrategia es idempotente: repetir una fecha produce el mismo estado logico. El proceso compara montos y cantidades entre fuente, staging y fact antes de confirmar. La carga delta reprocesa hoy y ayer para absorber correcciones de una fuente cuya hora no es confiable.

Justificacion para el jurado

El delta no depende de una marca horaria poco confiable. Reprocesar una ventana corta por fecha prioriza consistencia sobre una falsa precision temporal y mantiene el costo acotado.

2.4 Analitica de productos y clientes

El ABC debe calcularse dentro de un horizonte de analisis. Para decisiones operativas se recomienda 90 o 180 dias; para planeamiento, doce meses moviles. El historico completo es util como comparacion, pero sesga la clasificacion hacia productos antiguos o descatalogados. El RFM tambien depende del periodo y de los filtros de canal o tienda.

3. Decisiones tecnologicas y alternativas

Las decisiones se evaluaron por adecuacion al problema, costo, mantenibilidad, experiencia disponible y ruta de crecimiento. Elegir tecnologia no significa que las alternativas sean incorrectas, sino que ofrecen menor ajuste al contexto actual.

Decision	Por que se elige	Alternativa y trade-off
Python + pandas	Ecosistema ETL, tratamiento tabular, rapidez de desarrollo y librerias abiertas [5].	Java/Spark: mayor robustez distribuida, pero complejidad excesiva para el volumen actual.
FastAPI	API-first, validacion Pydantic, OpenAPI y Swagger automaticos [7].	Django: mas completo para aplicaciones monoliticas, pero mayor superficie para una API analitica.
PostgreSQL	ACID, integridad, ventanas, indices, materializacion y licencia abierta [6].	SQL Server: continuidad con la fuente, pero mayor acoplamiento y posible costo de licencia.
React + TypeScript	Componentes reutilizables y tipado para dashboards [8].	Angular: estructura mas prescriptiva, con mayor curva y peso para este alcance.
Vite	Ciclo de desarrollo rapido y build optimizado de activos estaticos [9].	Webpack manual: flexible, pero requiere mas configuracion y mantenimiento.
Docker	Empaquetado reproducible, aislamiento y portabilidad [10].	Instalacion nativa: menos capas, pero mayor deriva entre ambientes.

Decision	Por que se elige	Alternativa y trade-off
Docker Compose	Define servicios, redes y volúmenes en un YAML; adecuado para un host [11].	Kubernetes: alta disponibilidad y orquestación, no justificadas por la escala actual.
Nginx	Punto único para HTTPS, proxy, archivos estáticos y límites [12].	Exponer Uvicorn: más simple, pero sin capa edge ni terminación TLS.

Tabla 3. Registro resumido de decisiones arquitectónicas.

3.1 Por que un servidor Linux y no un escritorio remoto

El término correcto es servidor Linux o VPS. La administración se realiza por SSH; un escritorio gráfico consume memoria, amplía la superficie de ataque y no aporta valor al runtime. Docker Engine y Compose funcionan como servicios del sistema. El despliegue consiste en autenticar el registro, descargar imágenes versionadas, aplicar migraciones y levantar el stack.

3.2 Docker Hub como registro

Docker Hub permite distribuir imágenes inmutables en lugar de copiar código al servidor. El plan Personal incluye un repositorio privado y tiene límites de pulls; dos imágenes privadas separadas pueden requerir Docker Pro o un registro alternativo. En junio de 2026 Docker Pro figura en USD 11/mes [14]. Las imágenes deben publicarse sin secretos y referenciarse por tag versionado o digest.

4. Seguridad, normas y brechas

Declaración correcta

El proyecto está alineado parcialmente con buenas prácticas de ISO/IEC 27001:2022, NIST CSF 2.0, OWASP ASVS 5.0.0 y CIS Docker Benchmark. No está certificado ni puede afirmarse cumplimiento total sin SGSI, evaluación formal, evidencia y auditoría.

ISO/IEC 27001 define requisitos para un sistema de gestión de seguridad de la información que abarca personas, procesos y tecnología [1]. NIST CSF 2.0 organiza la gestión del riesgo en Govern, Identify, Protect, Detect, Respond y Recover [2]. OWASP ASVS proporciona requisitos verificables para controles técnicos de aplicaciones web [3]. CIS publica una guía de configuración segura para Docker [4].

4.1 Controles implementados

Dominio	Control actual	Evidencia	Estado
Acceso	JWT, roles, permisos y API keys con scope	security.*, PermissionChecker, api_clients	PARCIAL
Credenciales	Passwords hasheadas; API keys solo como hash	pwdlib y SHA-256 de claves aleatorias	IMPLEMENTADO
Datos fuente	Usuario de consulta sin modificación	Conexión ICG separada	IMPLEMENTADO
Integridad	Conciliación fuente-staging-fact	Controles de monto y cantidad	IMPLEMENTADO
Auditoría	Trace ID y redacción de secretos	AuditMiddleware y sanitizer	PARCIAL

Dominio	Control actual	Evidencia	Estado
Segregacion	ETL, seguridad y analytics con permisos	coolbox.etl.execute / analytics.read	IMPLEMENTADO
Cadena de suministro	Versiones Python fijadas y package-lock	requirements.txt / package-lock.json	PARCIAL
Despliegue	Red y volúmenes Compose	compose de produccion	OBJETIVO

Tabla 4. Mapeo de controles actuales. Estado cualitativo, no auditoria.

4.2 Brechas antes de produccion

Prioridad	Brecha	Tratamiento propuesto	Evidencia de cierre
ALTO	Registro de usuarios actualmente publico	Restringir a Admin o flujo de invitacion	Prueba 403 sin permiso
ALTO	JWT en localStorage y sin refresh token	Cookie HttpOnly/Secure, access corto y rotacion	Pruebas login-refresh-logout
ALTO	Sin HTTPS en arquitectura actual	Nginx + certificado + redireccion HTTP	Escaneo TLS y evidencia navegador
ALTO	Sin backup/restore probado	Backup cifrado externo y simulacro mensual	Acta de restauracion y RPO/RTO
ALTO	Sin rate limiting	Limites por IP, usuario y API key	Prueba de respuesta 429
MEDIO	Contenedores ejecutan como root	Usuario no privilegiado y filesystem restringido	docker inspect / Dockerfile
MEDIO	Sin CI/CD ni escaneo de imagen	Build, tests, SBOM, scan y push por tag	Pipeline exitoso y reporte
MEDIO	Cobertura de pruebas limitada	Integracion ETL, seguridad y contratos	Reporte de cobertura
MEDIO	Sin monitoreo ni alertas	Metricas, logs centralizados y healthchecks	Dashboard y alerta de prueba

Tabla 5. Backlog minimo de endurecimiento.

4.3 Objetivo de seguridad defendible

- Objetivo inicial: OWASP ASVS nivel 1 completo y controles seleccionados de nivel 2 para autenticacion.
- Aplicar principio de minimo privilegio en fuente, warehouse, API keys y contenedores.
- Definir inventario de activos, riesgos, responsables y tratamiento, alineado a NIST Govern/Identify.
- Documentar respuesta a incidentes, backups, RPO y RTO, alineado a Detect/Respond/Recover.
- No almacenar secretos en Docker Hub, imagenes, repositorios ni documentos.

5. Arquitectura de despliegue

Arquitectura objetivo de despliegue

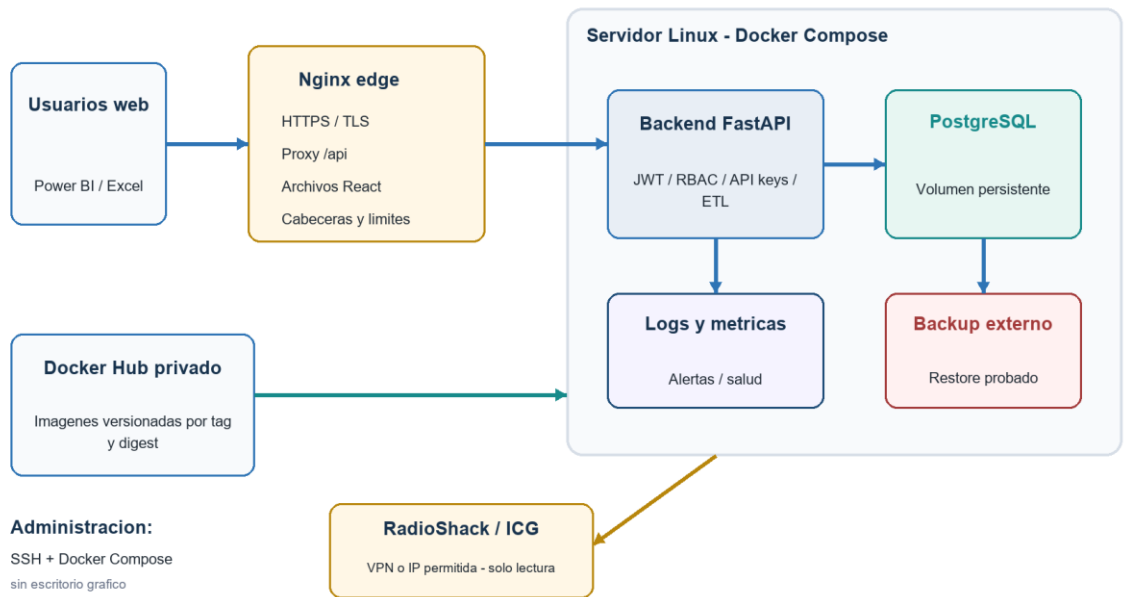


Figura 2. Despliegue objetivo en un servidor Linux con Docker Compose.

Solo Nginx publica 80/443. FastAPI y PostgreSQL permanecen en redes internas. La base no expone 5432 a Internet. La conexión a RadioShack requiere VPN, red corporativa o IP pública autorizada y credenciales de solo lectura. Los backups deben salir del mismo host para evitar que una falla del servidor destruya datos y copias.

5.1 Servicios propuestos en Compose

Servicio	Funcion	Exposicion	Persistencia
nginx	TLS, frontend, proxy /api y limites	80/443	Certificados/config
backend	API, analytics y ETL	Solo red interna	Sin estado
postgres	Warehouse y seguridad	Solo red interna	Volumen + backup
backup	Dump y copia externa programada	Sin puerto	Destino externo
monitoring (opcional)	Metricas, logs y alertas	VPN o interno	Segun herramienta

Tabla 6. Servicios y fronteras de exposicion.

5.2 Flujo de despliegue

1. Ejecutar tests y construir imágenes con tags inmutables, por ejemplo 1.0.0.
2. Escanear dependencias e imágenes; generar SBOM.

3. Publicar backend y frontend/edge en registro privado.
4. Conectar por SSH al servidor y descargar las imagenes.
5. Ejecutar alembic upgrade head como tarea controlada.
6. Levantar Compose y esperar healthchecks.
7. Ejecutar smoke tests de login, analytics y conectividad ICG.
8. Conservar la version anterior para rollback.

6. Capacidad, costos y rentabilidad

6.1 Estimacion inicial de recursos

Componente	RAM normal estimada	Pico / observacion
Nginx edge + frontend	40-120 MB	Depende de conexiones y cache
FastAPI 1 worker	250-500 MB	Incluye pandas, numpy y drivers
PostgreSQL	600 MB-2 GB	Cache, work_mem y consultas
ETL diario	0.5-3 GB adicionales	Depende de filas y columnas del dia
Sistema Linux	300-700 MB	Docker, kernel y agentes

Tabla 7. Estimaciones orientativas; deben reemplazarse por medicion.

Un host de 2 vCPU y 4 GB puede servir como piloto si el ETL se ejecuta sin alta concurrencia y el volumen diario es moderado. Para reducir riesgo de out-of-memory durante pandas, 4 vCPU y 8 GB es una base mas comoda. El disco debe dimensionarse como minimo para datos activos, indices, temporales y dos copias de respaldo; una regla inicial es tres veces el tamano actual medido.

6.2 Escenarios de costo

Costo mensual referencial de infraestructura

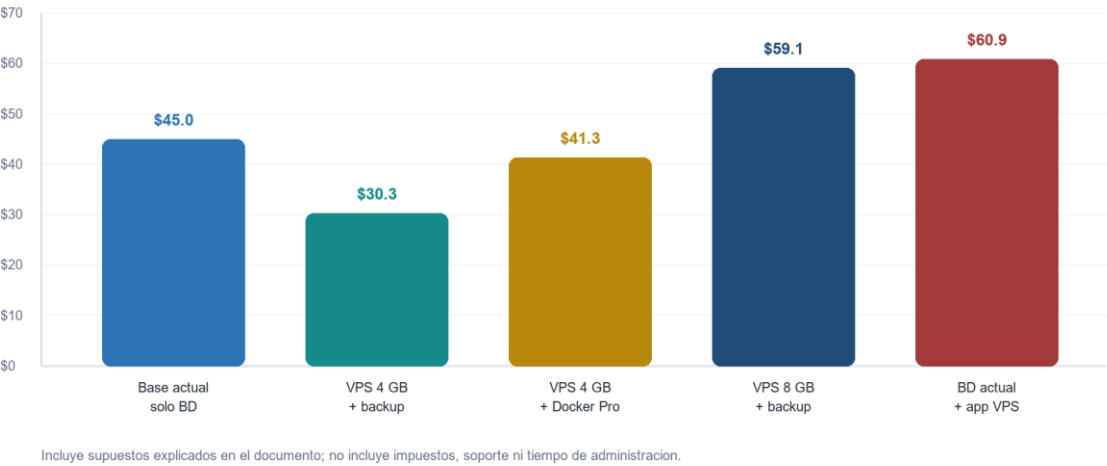


Figura 3. Comparacion mensual referencial, precios consultados el 28/06/2026.

Escenario	Supuesto mensual	Total aprox.	Uso recomendado
Actual	Base cloud reportada por el autor	USD 45	Desarrollo; costo alto si solo cubre BD
Piloto unico	VPS 4 GB USD 24 + backup 20% + dominio USD 1.5	USD 30.3	Demo remota y pocos usuarios
Piloto privado	Piloto + Docker Pro USD 11	USD 41.3	Dos imagenes privadas en Docker Hub
Host con margen	VPS 8 GB USD 48 + backup 20% + dominio	USD 59.1	ETL mas grande y mayor estabilidad
Separado	BD actual USD 45 + VPS 2 GB USD 12 + backup/dominio	USD 60.9	DB administrada y app separada

Tabla 8. No incluye impuestos, soporte, egress ni trabajo operativo.

DigitalOcean publica USD 24/mes para 2 vCPU, 4 GB y 80 GB SSD, y USD 48/mes para 4 vCPU, 8 GB y 160 GB SSD; el backup semanal porcentual se publica al 20% del costo del servidor [13]. Son referencias comparables, no una recomendacion contractual.

6.3 Rentabilidad y punto de equilibrio

La plataforma no genera ingresos por si sola; crea ahorro y capacidad de decision. La forma defendible de cuantificarla es medir horas evitadas, errores reducidos y decisiones comerciales mejoradas.

Formula

Beneficio mensual = horas ahorradas x costo-hora cargado + costo de errores evitados + margen incremental atribuible. ROI = (beneficio - costo) / costo.

Escenario	Supuesto de ahorro	Beneficio	Infra	Resultado
Conservador	1 analista x 6 h x USD 6	USD 36	USD 30	USD 6 netos; 1.2x
Base	2 analistas x 8 h x USD 8	USD 128	USD 41	USD 87 netos; 3.1x
Alto	3 analistas x 12 h x USD 10	USD 360	USD 60	USD 300 netos; 6.0x

Tabla 9. Ejemplos para la metodologia; reemplazar por datos reales.

Con un costo de USD 41/mes y un costo-hora de USD 8, el punto de equilibrio es 5.1 horas ahorradas por mes. Esta cifra es facil de validar durante un piloto comparando el tiempo anterior y posterior para preparar el mismo reporte.

7. Escalabilidad y roadmap

Ruta de escalamiento basada en evidencia

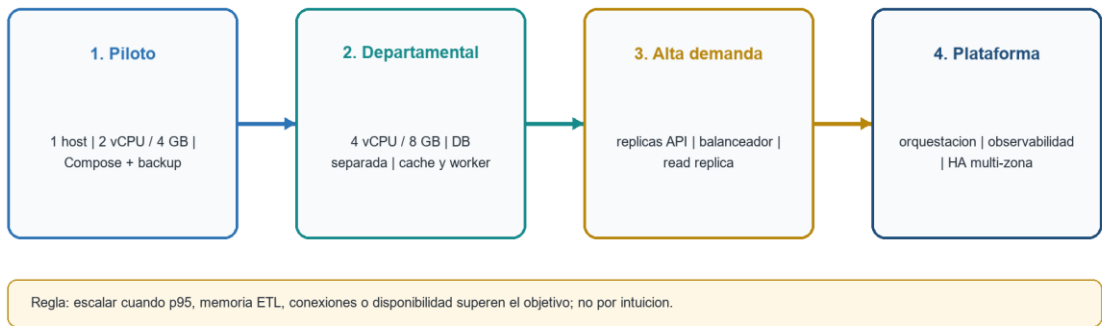


Figura 4. Escalamiento progresivo sin sobrediseñar.

7.1 Indicadores que disparan escalamiento

Metrica	Objetivo inicial	Accion si se supera
Latencia API p95	< 2 s en dashboard	EXPLAIN, agregados, cache; luego mas CPU
Memoria ETL	< 70% del host	Chunks, tipos compactos o mas RAM
CPU sostenida	< 70% fuera del batch	Optimizar consulta o separar worker
Conexiones DB	< 70% del limite	Ajustar pool, cache o separar cargas
Disponibilidad	>= 99% piloto	Healthchecks, alertas, replicas
RPO / RTO	24 h / 4 h piloto	Backups mas frecuentes y restore automatizado

Tabla 10. Metricas de capacidad propuestas.

7.2 Roadmap recomendado

Fase	Entregables	Criterio de salida
0. Titulacion	Arquitectura, demo local, pruebas, costos y riesgos	Sustentacion reproducible
1. Hardening	HTTPS, registro cerrado, refresh token, rate limit, backup	Checklist critico cerrado
2. Piloto	Servidor Linux, Compose, dominio, monitoreo basico	2-4 semanas sin incidentes
3. Optimizacion	EXPLAIN, agregado diario, cache de filtros, chunks	p95 y RAM dentro de objetivo
4. Escala	DB separada, worker, replicas y observabilidad	Demanda medible lo justifica

Tabla 11. Roadmap incremental.

7.3 Mejoras tecnicas priorizadas

- Crear agg_ventas_diarias y actualizarla al terminar cada fecha ETL.
- Ejecutar ABC y RFM sobre agregados o vistas materializadas cuando el volumen lo requiera.
- Procesar extracciones grandes por chunks para acotar memoria.
- Particionar fact_ventas por mes cuando la medicion demuestre beneficio.
- Separar ETL en un worker programado para no competir con la API.
- Agregar cache de filtros y KPIs con TTL corto.
- Incorporar pruebas de carga con volumen representativo.

8. Plan de sustentacion

8.1 Narrativa de 10 minutos

Minuto	Mensaje	Evidencia
0-1	Problema y restriccion de solo lectura	Contexto de RadioShack
1-3	Flujo ETL, staging y modelo dimensional	Figura 1 y conciliaciones
3-5	Analytics y decisiones de tecnologia	Endpoints, ABC/RFM y Tabla 3
5-7	Seguridad y consumo web/BI	JWT, RBAC, API keys y brechas
7-9	Despliegue, costo y escalabilidad	Figuras 2-4
9-10	Valor, limites y siguiente fase	ROI y roadmap

Tabla 12. Guion sugerido de exposicion.

8.2 Preguntas probables del jurado

¿Por que copiar los datos?

Para desacoplar analitica del sistema operacional, aplicar calidad y evitar impacto sobre la fuente.

¿Por que no consultar SQL Server directamente?

La fuente es de solo lectura, operacional y no controla las reglas comunes del dashboard.

¿Por que reprocesar hoy y ayer?

La hora no es confiable; una ventana por fecha absorbe correcciones con costo predecible.

¿Por que PostgreSQL?

Integridad ACID, SQL analitico, indices, materializacion, extensibilidad y ausencia de licencia propietaria.

¿Por que no microservicios?

El dominio y equipo no justifican costos distribuidos; el monolito modular conserva limites claros.

¿Por que Docker Compose y no Kubernetes?

Hay un host y baja escala; Compose aporta reproducibilidad sin el costo operativo de un cluster.

¿Cumple ISO 27001?

No se afirma certificacion. Se demuestra alineacion parcial y un backlog para un SGSI y verificacion formal.

¿Como se recupera de una falla?

Objetivo: backup externo, restore probado, imagen anterior y migracion reversible; aun es brecha de produccion.

¿Como escala?

Primero agregados, cache y chunks; luego separacion de DB/worker y finalmente replicas si las metricas lo exigen.

¿Es rentable?

Se calcula por horas y errores evitados; el piloto mide el antes/despues y valida el punto de equilibrio.

8.3 Demostracion sugerida

1. Mostrar el login y el usuario actual.
2. Ejecutar filtros de fecha, tienda y canal.
3. Comparar KPI, evolucion y top productos.
4. Mostrar ABC para 12 meses y luego para una tienda.
5. Crear una API key y explicar que el secreto solo se muestra una vez.
6. Consultar un endpoint con X-API-Key desde Power Query o Postman.
7. Mostrar una ejecucion delta y sus conteos de control.
8. Cerrar con docker compose config y el diagrama objetivo, sin depender de Internet.

9. Criterios de aceptacion para produccion

Area	Criterio de salida
Seguridad	HTTPS, registro restringido, refresh token seguro, rate limit y secretos fuera de imagen
Datos	Migraciones aplicadas, conciliaciones, Decimal y politicas de retencion
Continuidad	Backup externo, restauracion probada, RPO/RTO y rollback
Operaciones	Healthchecks, alertas, logs, espacio en disco y rotacion
Calidad	Tests unitarios, integracion, contratos, carga y seguridad
Despliegue	Imágenes privadas/versionadas, SBOM, scan y CI/CD
Red	Solo 80/443, PostgreSQL privado, VPN/IP permitida a ICG
Gobierno	Responsables, inventario, riesgos, accesos y evidencia

Tabla 13. Definition of Done para autorizar produccion.

Decision final sugerida

El proyecto esta listo para sustentarse como prototipo funcional y arquitectura objetivo. Antes de exponerlo a Internet, ejecutar la fase de hardening y un piloto controlado. La produccion es una decision posterior basada en uso, costo y riesgo, no un requisito para validar la arquitectura.

Referencias

- [1] [ISO/IEC 27001:2022 - Information security management systems.](#)
- [2] [NIST Cybersecurity Framework 2.0.](#)
- [3] [OWASP Application Security Verification Standard 5.0.0.](#)
- [4] [CIS Docker Benchmark.](#)
- [5] [Python Software Foundation - About Python.](#)
- [6] [PostgreSQL - About.](#)
- [7] [FastAPI - Features.](#)
- [8] [React - Quick Start.](#)
- [9] [Vite - Why Vite.](#)
- [10] [Docker - What is Docker?](#)
- [11] [Docker Compose documentation.](#)
- [12] [Nginx official site and documentation.](#)
- [13] [DigitalOcean Droplet Pricing, consulted 2026-06-28.](#)
- [14] [Docker plans and Docker Hub usage limits, consulted 2026-06-28.](#)

Anexo A. Evidencias técnicas actuales

Evidencia	Estado observado
Backend	FastAPI, SQLAlchemy, Alembic, pandas y PostgreSQL
Frontend	React 19, TypeScript, Vite 8, Zustand y Recharts
ETL	Fecha, rango, delta, staging, controles y carga por lotes
Analytics	Ventas, productos, clientes, ABC, RFM y filtros
Seguridad	JWT, RBAC, API clients, hash, scope y auditoria redactada
Pruebas	14 pruebas unitarias aprobadas al cierre de la revision
Migraciones	Una cabeza Alembic: d91a0d67c8fe
Contenedores	Dockerfiles y Compose validos; falta stack unificado con Nginx

Tabla 14. Evidencia obtenida del código local revisado.

Anexo B. Supuestos y límites

- Las cifras de RAM son estimaciones y deben validarse con docker stats y una carga representativa.
- Los costos no incluyen impuestos, soporte, egress, dominio real ni horas de administracion.
- La disponibilidad no puede garantizarse en un unico host.
- El acceso remoto a RadioShack depende de conectividad autorizada fuera del código.
- La rentabilidad debe reemplazar los ejemplos por salarios, tiempos y errores reales.
- La alineacion normativa no equivale a certificacion ni auditoria independiente.